INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

APLICACIONES PARA COMUNICACIONES EN RED

Reporte

“Chat”

Grupo: 6CM2

Integrantes

- García Escamilla Bryan Alexis
- Meléndez Macedonio Rodrigo

Profesor

Moreno Cervantes Axel Ernesto

Fecha de entrega: 14 de noviembre de 2025

ÍNDICE

INTRODUCCIÓN 2
DESARROLLO 4
1.- Archivo: utils.py 4
2.- Archivo: audio.py 5
3.- Archivo: stickers.py 11
4.- Archivo: servidor.py 14
5.- Archivo: cliente.py 21
CONCLUSIÓN 28

INTRODUCCIÓN

Hilo (Thread)

Un hilo de ejecución (thread) es la unidad más pequeña de procesamiento que puede realizar un sistema operativo.

Dentro de un programa, un hilo representa una secuencia individual de instrucciones que se ejecuta de forma independiente, aunque comparte el mismo espacio de memoria y los mismos recursos del proceso al que pertenece.

un proceso puede contener uno o varios hilos que trabajan de manera concurrente. Cada hilo puede realizar una tarea diferente al mismo tiempo, lo que permite dividir el trabajo en partes simultáneas, mejorando la eficiencia, la capacidad de respuesta, o el rendimiento de un programa.

Características principales

- Comparte memoria y recursos con los demás hilos del mismo proceso.
- Se ejecuta en paralelo o concurrentemente con otros hilos.
- Posee su propio contador de programa, pila de ejecución y registros.
- Requiere menos recursos para crearse y gestionarse que un proceso independiente.

¿Por qué son útiles?

- ✓ Para realizar tareas en segundo plano sin bloquear la interfaz.
- ✓ Para procesar múltiples tareas simultáneamente.
- ✓ Para aprovechar múltiples núcleos del CPU.

Biblioteca “threading”

La biblioteca threading en Python proporciona una interfaz de alto nivel para trabajar con hilos de ejecución. Permite crear, iniciar, sincronizar y coordinar hilos, facilitando el desarrollo de programas que necesitan realizar múltiples tareas de forma simultánea sin bloquear la ejecución principal.

Comandos

Elemento	Tipo	Descripción
Thread	Clase	Permite crear y administrar un hilo de ejecución.
start()	Método de Thread	Inicia la ejecución del hilo.
run()	Método de Thread	Contiene el código que el hilo ejecutará. Puede sobrescribirse.
join()	Método de Thread	Hace que el programa espere a que el hilo termine antes de continuar.
is_alive()	Método de Thread	Indica si el hilo sigue en ejecución.
current_thread()	Función	Devuelve el objeto del hilo actual que se está ejecutando.

active_count()	Función	Retorna la cantidad de hilos activos en ese momento.
Lock	Clase	Proporciona un mecanismo para evitar que múltiples hilos accedan a recursos compartidos simultáneamente.
acquire()	Método de Lock	Bloquea el recurso para un solo hilo.
release()	Método de Lock	Libera el recurso para que otros hilos puedan usarlo.
RLock	Clase	Similar a Lock, pero permite que el mismo hilo lo adquiera varias veces sin bloquearse.
Semaphore	Clase	Controla el acceso a un recurso mediante contadores, útil cuando varios hilos pueden acceder en grupo.
Event	Clase	Permite que los hilos esperen hasta que ocurra una señal o evento.
Condition	Clase	Facilita la comunicación entre hilos que esperan ciertos estados o condiciones.

Biblioteca “sounddevice”

sounddevice es una biblioteca de Python que permite reproducir y grabar audio en tiempo real mediante el uso del sistema de audio del dispositivo. Está construida sobre la API PortAudio, lo que la hace multiplataforma (Windows, Linux y macOS).

Características

- Funciona en Windows, Linux y macOS
- El audio se maneja como arreglos NumPy
- Permite grabación y reproducción simultáneas
- Soporta configuraciones de audio profesional (48kHz, 96kHz, etc.)

Comandos

Función	Descripción
sd.play(data, samplerate)	Reproduce audio desde un arreglo NumPy.
sd.wait()	Espera a que finalice la reproducción o grabación.
sd.rec(frames, samplerate, channels)	Graba audio y lo devuelve como arreglo NumPy.
sd.stop()	Detiene la reproducción o grabación.
sd.query_devices()	Muestra los dispositivos de audio disponibles.
sd.default.device	Seleccionar dispositivo de entrada y salida.

Biblioteca “wavio”

wavio es una biblioteca de Python diseñada para leer y escribir archivos de audio en formato WAV, usando internamente NumPy para manejar los datos. Es ligera y se utiliza principalmente para guardar grabaciones en disco.

Características

Se enfoca únicamente en archivos WAV

Maneja las muestras como arreglos NumPy

Permite especificar resolución (8, 16, 24, 32 bits)

Función	Descripción
wavio.write(filename, data, samplerate, sampwidth)	Guarda un arreglo NumPy como archivo WAV
wavio.read(filename)	Carga un archivo WAV y devuelve un objeto con: .data, .rate, .sampwidth

DESARROLLO

Para chat se crearon 5 archivos: “utils.py”, “audio.py”, “stickers.py”, “servidor.py” y “cliente.py”, donde los primeros tres son archivos de apoyo que contienen variables globales, funciones e interfaces gráficas programadas para tener más limpio y ordenado el código. Por lo tanto, para el caso de estos archivos no se va a realizar una explicación muy detallada de su contenido y solo se mencionarán estos de manera breve.

1.- Archivo: utils.py

- **limpiar_terminal()**

```python
import os

# Colores ANSI
RED = "\033[91m"
GREEN = "\033[92m"
YELLOW = "\033[93m"
BLUE = "\033[94m"
MAGENTA = "\033[95m"
ORANGE = "\033[33m"
RESET = "\033[0m"

def limpiar_terminal() -> None:
    os.system('cls' if os .name == 'nt' else 'clear')
```

Imagen 1. Variables globales para colores personalizados y función para limpiar la terminal “limpiar_terminal()”.

Parámetros:

- **No hay parámetros para esta función.**

Descripción de variables: Para este bloque de código se tienen las variables globales RED, GREEN, YELLOW, BLUE, MAGENTA, ORANGE y RESET, las cuales nos van a servir para personalizar los colores de las letras dentro de la terminal con el fin de hacer más entendible el programa.

Descripción de la función: También se tiene la función **"limpiar_terminal()"**, que se encarga de limpiar la terminal cuando sea necesario hacerlo dentro del programa.

2.- Archivo: audio.py

- **enviar_audio_gbn(**server_addr_main: tuple, metadata: dict, audio_bytes: bytes, window_size=6, timeout=3.0**)**

```python
import tkinter as tk
from tkinter import ttk
from datetime import datetime
from pathlib import Path
import socket
import struct
import json

import pygame
import sounddevice as sd

import utils

pygame.mixer.init()

ENCABEZADO_FORMAT = "!I"
ENCABEZADO_SIZE = 4
MAX_BYTES_PAQUETE = 1024

def enviar_audio_gbn(server_addr_main: tuple, metadata: dict, audio_bytes: bytes, window_size=6, timeout=3.0) -> str | None:
    tamano_archivo = len(audio_bytes)
    total_paquetes = (tamano_archivo + (MAX_BYTES_PAQUETE - ENCABEZADO_SIZE) - 1) // (MAX_BYTES_PAQUETE - ENCABEZADO_SIZE)

    metadata_enviar = dict(metadata)
    metadata_enviar.update({"tipo": "AUDIOINFO",
                            "tamano": tamano_archivo,
                            "total_paquetes": total_paquetes})

    temp_sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    temp_sock.bind(('', 0))
    temp_sock.sendto(json.dumps(metadata_enviar).encode(), server_addr_main)
```

Imagen 2. Primera parte del código que corresponde a la función "enviar_audio_gbn()".

```python

    temp_sock.settimeout(3.0)
    try:
        data, _ = temp_sock.recvfrom(4096)
    except socket.timeout:
        print("No se recibió READY del servidor (timeout).")
        temp_sock.close()
        return None

    try:
        respuesta = json.loads(data.decode())
    except:
        print("Respuesta inválida del servidor al solicitar transferencia de audio.")
        temp_sock.close()
        return None

    if respuesta.get("tipo") != "READY" or "port" not in respuesta:
        print("Servidor no listo para transferencia.")
        temp_sock.close()
        return None

    audio_port = int(respuesta["port"])
    server_ip = server_addr_main[0]
    server_audio_addr = (server_ip, audio_port)

    sock_audio = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    sock_audio.bind(('', 0))
    sock_audio.settimeout(timeout)

    print(f">> Enviando audio a {server_audio_addr} — {tamano_archivo} bytes, {total_paquetes} paquetes")
```

Imagen 3. Segunda parte del código que corresponde a la función “enviar_audio_gbn()”.

```python

    base = 0
    next_seq = 0
    payload_size = MAX_BYTES_PAQUETE - ENCABEZADO_SIZE

    while base < total_paquetes:
        while next_seq < base + window_size and next_seq < total_paquetes:
            inicio = next_seq * payload_size
            fin = inicio + payload_size
            chunk = audio_bytes[inicio:fin]
            encabezado = struct.pack(ENCABEZADO_FORMAT, next_seq)
            paquete = encabezado + chunk
            sock_audio.sendto(paquete, server_audio_addr)
            next_seq += 1

        try:
            data, _ = sock_audio.recvfrom(4096)
            msj = json.loads(data.decode())
            if msj.get("tipo") == "ACK":
                ack_seq = int(msj["num_seq"])
                if ack_seq >= base:
                    base = ack_seq + 1
        except socket.timeout:
            next_seq = base
            print(f">> Timeout — reintentando desde paquete {base}")

    sock_audio.sendto(b"FIN", server_audio_addr)
    sock_audio.close()
    temp_sock.close()

    return metadata.get("nombre")
```

Imagen 4. Tercera parte del código que corresponde a la función “enviar_audio_gbn()”.

Parámetros:

- **server_addr_main: tuple**
  - (ip, puerto) del servidor principal, donde se envía el AUDIOINFO.
- **metadata: dict**
  - diccionario con metadatos (incluye 'nombre', 'frecuencia', 'canales', 'privado', 'to', 'user', 'sala', etc.).
- **audio_bytes: bytes**
  - bytes del audio (int16).
- **window_size: int**
  - Número de paquetes que se pueden enviar al mismo tiempo.
- **timeout: float**
  - Tiempo máximo de espera para recibir mensajes del servidor.

Descripción de la función: Esta función contiene una parte del algoritmo Go-Back-N (GBN) visto y usado durante el desarrollo de la practica No. 2, el cual se encarga de fragmentar un archivo de audio en N paquetes para evitar la pérdida de estos durante el envío del archivo hacia el destino, sin embargo, en caso de que se pierda un paquete, el algoritmo se encarga de retroceder N veces hasta que el paquete sea enviado correctamente al destino, asegurando la transmisión completa del archivo.

Bajo esta misma lógica, implementamos el algoritmo a esta práctica de chat entre diferentes clientes para que estos puedan enviar audios entre sí, evitando la perdida de paquetes durante la transmisión del archivo a determinado destino. Al tratarse del mismo código utilizado anteriormente, se omitirá la explicación a profundidad del código, sin embargo, algo que si es importante mencionar es que en esta parte del código solo se envían los paquetes del archivo del audio hacia el servidor, pero esto se explicará más adelante.

Clase <u>Audio</u>:

- **grabar_audio(**self, mensaje: dict, sala: str, usuario: str**)**

```python
class Audio:
    def grabar_audio(self, mensaje: dict, sala: str, usuario: str) -> str | None:
        fecha = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
        nombre_audio = f"{usuario}_{fecha}.wav"

        privado = bool(mensaje.get("privado", False))
        dest = mensaje.get("to")
        grab_name = nombre_audio

        duracion = 5
        frecuencia = 44100
        canales = 2

        print(f">> {utils.RED}[system]{utils.RESET} Grabando audio...")
        audio = sd.rec(int(duracion * frecuencia), samplerate=frecuencia, channels=canales, dtype="int16")
        sd.wait()
        print(f">> {utils.RED}[system]{utils.RESET} Grabación completa")

        try:
            audio_bytes = audio.tobytes()
        except Exception:
            import numpy as np
            audio_bytes = np.asarray(audio, dtype="int16").tobytes()

        metadata = {"user": usuario,
                    "from": usuario,
                    "to": dest,
                    "sala": sala,
                    "privado": privado,
                    "nombre": grab_name,
                    "frecuencia": frecuencia,
                    "canales": canales}

        SERVER = ("127.0.0.1", 5007)
        enviar_nombre = enviar_audio_gbn(SERVER, metadata, audio_bytes, window_size=6, timeout=3.0)

        return enviar_nombre
```

Imagen 5. Código que corresponde a la función “grabar_audio()”.

Parámetros:

- **self**
- **mensaje: dict**
  - Diccionario con los metadatos para la grabación.
- **sala: str**
  - Nombre de la sala desde donde se graba el audio.
- **usuario: str**
  - Nombre del usuario que graba el audio.

Descripción de la función: Esta función se encarga de grabar el audio en tiempo real con ayuda de la biblioteca “sounddevice”. Cuando uno de los clientes lo solicite dentro del chat, esta función grabará el audio desde el micrófono del dispositivo durante 5 segundos y terminado el tiempo, se guardara toda la metainformación necesaria para identificar fácilmente que es lo que se quiere hacer con dicho audio.

Como se puede ver en la imagen, al final se manda a llamar la función “enviar_audio_gbn()” para asegurar la transmisión completa del audio hacia el destinatario y evitar perdida de

paquetes durante el envío. Además, aquí es importante mencionar que cada vez que un cliente graba un audio, este se guarda bajo un nombre que contiene el nombre del cliente que está grabando el audio seguido de la fecha y hora en la que esta grabando el audio. Esto se hizo así para evitar que los archivos no se sobrescribieran bajo el mismo nombre y de esta manera se pueda tener un registro completo de todos los audios que graba el usuario dentro de una sala.

- **buscar_audio(**self, carpeta_sala: Path, carpeta_personal: Path, usuario: str, nombre_audio: str**)**

```
133     def buscar_audio(self, carpeta_sala: Path, carpeta_personal: Path, usuario: str, nombre_audio: str) -> Path | None:
134         ruta_directa = carpeta_sala / nombre_audio
135         if ruta_directa.exists():
136             return ruta_directa
137
138         for archivo in carpeta_sala.rglob("*"):
139             if archivo.is_file:
140                 nombre = archivo.name
141                 if nombre.startswith(usuario) and nombre.endswith(nombre_audio):
142                     return archivo
143
144         ruta_personal = carpeta_personal / nombre_audio
145         if ruta_personal.exists():
146             return ruta_personal
147
148         return None
```

Imagen 6. Código que corresponde a la función “buscar_audio”.

Parámetros:

- **self**
- **carpeta_sala: Path**
  - Ruta hacia la carpeta de la sala donde se buscará el audio.
- **carpeta_personal: Path**
  - Ruta hacia la carpeta del usuario donde se buscará el audio.
- **usuario: str**
  - Nombre del usuario que quiere reproducir el audio.
- **nombre_audio: str**
  - Nombre del audio que se quiere buscar.

Descripción de la función: Esta función se encarga de buscar el audio enviado por uno de los clientes dentro de una sala. Aquí es importante mencionar que cada cliente cuenta con su propia carpeta al momento de entrar a una sala y en esta carpeta se almacenaran todos los audios que él va a grabar. Entendiendo eso, esta función se enfoca en buscar entre las diferentes carpetas creadas para cada cliente dentro de la sala, de esta forma es más fácil identificar los audios pues están clasificados por cliente.

- **reproducir_audio(**self, ruta_audio: Path, nombre_audio: str**)**

```python

    def reproducir_audio(self, ruta_audio: Path, nombre_audio: str) -> None:
        def reproducir() -> None:
            pygame.mixer.music.play()
            label_state.config(text="Reproduciendo audio...")

        def pausar() -> None:
            pygame.mixer.music.pause()
            label_state.config(text="Audio en pausa")

        def continuar() -> None:
            pygame.mixer.music.unpause()
            label_state.config(text="Continuando audio...")

        def detener() -> None:
            pygame.mixer.music.stop()
            label_state.config(text="Audio detenido")

        def ajustar_volumen(valor: int) -> None:
            volumen = float(valor)
            pygame.mixer.music.set_volume(volumen)
            label_volume.config(text=f"Volumen al {int(volumen * 100)}%")

        try:
            pygame.mixer.music.load(ruta_audio)
            archivo_cargado = True
        except Exception as e:
            archivo_cargado = False
            print(f"Error al cargar el archivo: {e}")

        root = tk.Tk()
        root.title("Reproductor")
        root.geometry("400x300")
        root.resizable(False, False)
```

Imagen 7. Primera parte del código que corresponde a la función “reproducir_audio()”.

```python

        if archivo_cargado:
            label = tk.Label(root, text=f"Audio cargado: \n{nombre_audio}", wraplength=380)
        else:
            label = tk.Label(root, text="No se pudo cargar el audio", fg="red")

        label.pack(pady=10)
        label_state = tk.Label(root, text="Reproductor inactivo", fg="gray")
        label_state.pack(pady=5)

        frame_btns = tk.Frame(root)
        frame_btns.pack(pady=15)

        btn_play = tk.Button(frame_btns, text="▶ Reproducir", command=reproducir, state="normal" if archivo_cargado else "disabled")
        btn_play.grid(row=0, column=0, padx=5)
        btn_pause = tk.Button(frame_btns, text="⏸ Pausar", command=pausar, state="normal" if archivo_cargado else "disabled")
        btn_pause.grid(row=0, column=1, padx=5)
        btn_continue = tk.Button(frame_btns, text="▶ Continuar", command=continuar, state="normal" if archivo_cargado else "disabled")
        btn_continue.grid(row=0, column=2, padx=5)
        btn_stop = tk.Button(frame_btns, text="⏹ Detener", command=detener, state="normal" if archivo_cargado else "disabled")
        btn_stop.grid(row=0, column=3, padx=5)

        label_volume = tk.Label(root, text="Volumen al 50%")
        label_volume.pack(pady=5)

        control_volume = ttk.Scale(root, from_=0, to=1, orient="horizontal", value=0.5, command=ajustar_volumen, length=250)  # type: ignore
        control_volume.pack()
        pygame.mixer.music.set_volume(0.5)

        root.mainloop()
```

Imagen 8. Segunda parte del código que corresponde a la función “reproducir_audio()”.

Parámetros:

- **self**
- **ruta_audio: Path**
    - Ruta del audio a reproducir.
- **nombre_audio: str**
    - Nombre del audio que se quiere reproducir.

Descripción de la función: En esta función, mediante el uso de las bibliotecas “tkinter” y “pygame” se definen todos los elementos para construir una interfaz gráfica de usuario simple para reproducir un audio en el momento en el que un cliente lo envía a través del chat. Del mismo modo que con el algoritmo Go-Back-N (GBN), este código es reutilizado de la practica No. 2 realizada anteriormente, por lo que se omitirá la explicación de esta parte del código.

Sin embargo, es importante mencionar que, dentro de esta función se definen otras funciones que ayudan en el funcionamiento de esta. A continuación, se listan las funciones utilizadas:

Función	Descripción
reproducir()	Comienza la reproducción del audio al presionar el botón 'Reproducir'.
pausar()	Pausa la reproducción del audio al presionar el botón 'Pausar'.
continuar()	Continua la reproducción del audio al presionar el botón 'Coninuar'.
detener()	Detiene la reproducción del audio al presionar el botón 'Detener'.
ajustar_volumen(valor: int)	**Parámetro** → valor: int → Volumen inicial del audio. Ajusta el volumen del audio.

3.- Archivo: stickers.py

- **Obtener_sticker(**nombre_sticker: str**)**

```python
def obtener_sticker(nombre_sticker: str) -> str:
    match nombre_sticker:
        case 'shrek':
            sticker = """
⠀⠀⠀
```

```
        case 'dog':
            sticker = """
 / \\__
(    @\\___
 /         O
/   (_____/
/_____/   U
"""
        case 'cat':
            sticker = """
 /\\_/\\
( o.o )
 > ^ <
"""
        case 'heart':
            sticker = """
  **      **
 *****   *****
**************
 ************
  **********
    ******
      **
"""
        case 'rocket':
            sticker = """
    ^
   /^\\
  /___\\
  |= |=
  |.-.|
  |'-'|
  |   |
 /|_|_|\\
   / \\
```

Imagen 10. Segunda parte del código que corresponde a la función “obtener_sticker()”, stickers de perro, gato, corazón y cohete.

```
 /___\\
"""
        case 'rabbit':
            sticker = """
 (\\_/)
 ( •_•)
 / >🍪
"""
        case 'owl':
            sticker = """
 ,_,
(O,O)
(   )
 " "
"""
        case 'dinosaur':
            sticker = """
            __
           / _)
   .-^^^-/ /
__/       /
<__.|_|-|_|
"""
        case _:
            return ""

    return sticker
```

Imagen 11. Tercera parte del código que corresponde a la función “obtener_sticker()”, stickers de conejo, búho y dinosaurio.

Parámetros:

- **nombre_sticker: str**
  - Nombre del sticker que se quier enviar.

Descripción de la función: En esta función se definen cada uno de los casos para cada sticker que puede utilizar cada cliente dentro del chat. Cada sticker está definido dentro de una cadena de caracteres para facilitar la implementación de estos dentro de la terminal. La función devuelve una cadena dependiendo del sticker que elija el cliente dentro del programa.

4.- Archivo: servidor.py

- **Enviar_unicast(**data: dict, addr: tuple**)**

```python
import socket
import json
import threading
import struct
import time
from pathlib import Path

import wave

import utils

SERVER_ADDR = ("0.0.0.0", 5007)
usuarios = {"general": {}}

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.bind(SERVER_ADDR)

print("Servidor de chat activo...")

ENCABEZADO_FORMAT = "!I"
ENCABEZADO_SIZE = 4
MAX_BYTES_PAQUETE = 1024

def enviar_unicast(data: dict, addr: tuple) -> None:
    sock.sendto(json.dumps(data).encode(), addr)

```

Imagen 12. Código que corresponde a la inicialización del servidor y a la función "enviar_unicast()".

Parámetros:

- **data: dict**
  Metadatos a enviar al usuario.
- **addr: tuple**
  - (direccion_IP, puerto)
    - **direccion_IP** (str): Dirección IP del usuario.
    - **puerto** (int): Puerto del usuario.

Descripción de variables: En esta parte del código primero se define la dirección del servidor y el puerto correspondiente a este dentro de una tupla, luego se crea el socket definiendo su familia y el tipo de socket en cuestión, que en este caso usaremos socket de datagrama. Enseguida, se inicializa el socket y finalmente se definen algunas variables globales que serán útiles para la recepción de los paquetes del archivo de audio usando el algoritmo Go-Back-N.

Descripción de la función: Esta función se encarga de enviar la información a un solo cliente con base en su dirección IP en caso de que los mensajes del chat sean privados.

- **Enviar_publico(**data: dict, sala: str**)**

```python

def enviar_publico(data: dict, sala: str) -> None:
    for usuario_addr in usuarios[sala].values():
        enviar_unicast(data, usuario_addr)
```

Imagen 13. Código que corresponde a la función “enviar_publico()”.

Parámetros:

- **data: dict**
  - Metadatos a enviar a cada usuario.
- **sala: str**
  - Sala a la que se envían los metadatos.

Descripción de la función: En esta función se usa un ciclo “for” para enviar la información a cada uno de los usuarios conectados en el chat en caso de que los mensajes del chat sean públicos.

- **recibir_audio_gbn(**audio_sock: sock.socket, cliente_addr: tuple, info: dict**)**

```python
def recibir_audio_gbn(audio_sock: socket.socket, cliente_addr: tuple, info: dict) -> None:
    nombre = info.get("nombre", f"{info.get('user','anon')}_{int(time.time())}.wav")
    tamano = int(info.get("tamano", 0))
    total_paquetes = int(info.get("total_paquetes", 0))
    frecuencia = int(info.get("frecuencia", 44100))
    canales = int(info.get("canales", 2))
    sala = info.get("sala", "general")
    privado = bool(info.get("privado", False))
    usuario = info.get("user") or info.get("from") or "anon"
    destinatario = info.get("to")

    print(f">> Iniciando recepción de audio '{nombre}' desde {cliente_addr} (sala='{sala}', user='{usuario}', privado={privado}, paquetes={total_paquetes})")
    paquetes = {}
    num_seq_esperado = 0
    audio_sock.settimeout(3.0)

    while True:
        try:
            paquete, addr = audio_sock.recvfrom(4096)
        except socket.timeout:
            continue

        if 'addr_audio' not in locals():
            addr_audio = addr

        if addr != addr_audio:
            continue

        if paquete == b"FIN":
            print("FIN recibido para transferencia de audio.")
            audio_sock.settimeout(None)
            break

        if len(paquete) < ENCABEZADO_SIZE:
            continue
```

Imagen 14. Primera parte del código que corresponde a la función “recibir_audio_gbn()”.

```python

        num_seq = struct.unpack(ENCABEZADO_FORMAT, paquete[:ENCABEZADO_SIZE])[0]
        datos = paquete[ENCABEZADO_SIZE:]

        if num_seq == num_seq_esperado:
            paquetes[num_seq] = datos
            num_seq_esperado += 1

            ack = {"tipo": "ACK",
                   "num_seq": num_seq}
            audio_sock.sendto(json.dumps(ack).encode(), addr_audio)
            print(f">> Recibido paquete {num_seq}, enviado ACK{num_seq}")
        else:
            ack_num = max(0, num_seq_esperado - 1)
            ack = {"tipo": "ACK",
                   "num_seq": ack_num}
            audio_sock.sendto(json.dumps(ack).encode(), addr_audio)
            print(f">> Paquete fuera de orden {num_seq}, reenvío ACK{ack_num}")

    audio_bytes = bytearray()
    for i in range(num_seq_esperado):
        audio_bytes.extend(paquetes.get(i, b""))

    audio_bytes = bytes(audio_bytes[:tamano])

    carpeta = Path(__file__).parent
    carpeta_sala = carpeta / f"{sala}"
    carpeta_sala.mkdir(parents=True, exist_ok=True)

    if privado and destinatario:
        carpeta_dest = carpeta_sala / f"{destinatario}"
        carpeta_dest.mkdir(parents=True, exist_ok=True)
        ruta_archivo = carpeta_dest / nombre
    else:
        ruta_archivo = carpeta_sala / nombre
```

Imagen 15. Segunda parte del código que corresponde a la función “recibir_audio_gbn()”.

```python

    try:
        with wave.open(str(ruta_archivo), "wb") as wf:
            wf.setnchannels(canales)
            wf.setsampwidth(2)
            wf.setframerate(frecuencia)
            wf.writeframes(audio_bytes)
        print(f"💾 Audio guardado en: {ruta_archivo}")
    except Exception as e:
        print(f"Error al guardar audio: {e}")
        return

    aviso = {"tipo": "audio",
             "sala": sala,
             "from": usuario,
             "nombre": nombre,
             "privado": privado,
             "content": f"{utils.GREEN}[🎙][{usuario}]{utils.RESET} ha enviado el audio '{nombre}'"}

    if privado and destinatario:
        if destinatario in usuarios.get(sala, {}):
            enviar_unicast(aviso, usuarios[sala][destinatario])
        else:
            error = {"tipo": "aviso",
                     "sala": sala,
                     "content": f">> {utils.RED}[system]{utils.RESET} Usuario '{destinatario}' no está conectado"}
            enviar_unicast(error, usuarios[sala].get(usuario, cliente_addr))
    else:
        enviar_publico(aviso, sala)

    try:
        audio_sock.close()
    except:
        pass
```

Imagen 16. Tercera parte del código que corresponde a la función “recibir_audio_gbn()”.

Parámetros:

- **audio_sock: sock.socket**
  Socket UDP ligado al puerto asignado para esta transferencia.
- **cliente_addr: tuple**
  - (direccion_IP, puerto)
    - **direccion_IP** (str): Dirección IP del cliente.
    - **puerto** (int): Puerto de origen del cliente.
- **info: dict**
  Metadatos enviados por el cliente (incluye nombre, sala, user, privado, to, frecuencia, canales, tamano, total_paquetes).

Descripción de la función: Esta función es la que se encarga de recibir los paquetes enviados por el cliente que haya enviado el audio. Si se recibieron todos los paquetes correctamente, entonces el audio se guarda en la ruta del cliente que haya grabado el audio para que así los otros clientes que estén conectados en la sala puedan acceder a dicho archivo. Por otro lado, en caso de que él envió haya sido hacia otro cliente de manera privada, entonces se realiza el unicast solo hacia ese cliente y nadie más que el podrá acceder a dicho audio.

- **manejar_cliente()**

```
139 def manejar_cliente() -> None:
140     while True:
141         data, addr = sock.recvfrom(4096)
142         try:
143             mensaje = json.loads(data.decode())
144         except:
145             continue
146
147         tipo = mensaje.get("tipo")
148         usuario = mensaje.get("user") or mensaje.get("from")
149         sala = mensaje.get("sala", "general")
150
151         if tipo == "listar_salas":
152             salas = list(usuarios.keys())
153             respuesta = {"tipo": "salas",
154                          "lista": salas}
155             enviar_unicast(respuesta, addr)
156             continue
157
158         if sala not in usuarios:
159             usuarios[sala] = {}
160
161         if tipo == "inicio":
162             if usuario not in usuarios[sala]:
163                 usuarios[sala][usuario] = addr
164                 aviso = {"tipo": "aviso",
165                          "sala": sala,
166                          "content": f">> {utils.GREEN}[+]{utils.RESET}{utils.BLUE}[{usuario}]{utils.RESET} se ha unido a la sala"}
167                 enviar_publico(aviso, sala)
168
169             usuarios_sala = {"tipo": "usuarios",
170                              "sala": sala,
171                              "lista": list(usuarios[sala].keys())}
172             enviar_publico(usuarios_sala, sala)
```

Imagen 17. Primera parte del código que corresponde a la función “manejar_cliente()”.

Parámetros:

- **No hay parámetros para esta función.**

Descripción: En esta parte del código primero se encuentra el bucle “while” principal que mantendrá el servidor abierto hasta que se indique lo contrario, por lo que en esta parte es donde se van a recibir todas las peticiones de los clientes para ser respondidas por el servidor. Dependiendo del tipo de información que le llegue al servidor, es como este responderá, y como primer caso tenemos el de listar salas, en el que simplemente devuelve al cliente mediante unicast la lista total de las salas creadas. Luego tenemos otro caso en el que se crea la sala si no existe.

Después tenemos una serie de condicionales anidados que corresponden a las acciones que puede realizar el cliente dentro de la sala y como primer caso tenemos cuando un usuario entra a una sala; en caso de que aún no haya entrado a la sala, se añadirá a la sala, enviara un aviso dentro de la sala a la que entro y se actualizara la lista de los usuarios dentro de dicha sala.

```
173
174         elif tipo == "msj":
175             if not mensaje.get("privado", False):
176                 enviar_publico(mensaje, sala)
177             elif mensaje.get("privado", False):
178                 dest = mensaje.get("to")
179                 if dest in usuarios[sala]:
180                     enviar_unicast(mensaje, usuarios[sala][dest])
181                 else:
182                     error = {"tipo": "aviso",
183                              "sala": sala,
184                              "content": f">> {utils.RED}[system]{utils.RESET} Usuario '{dest}' no está conectado"}
185                     enviar_unicast(error, addr)
186
187         elif tipo == "salir":
188             if usuario in usuarios[sala]:
189                 usuarios[sala].pop(usuario)
190                 aviso = {"tipo": "aviso",
191                          "sala": sala,
192                          "content": f">> {utils.RED}[-]{utils.RESET}{utils.BLUE}[{usuario}]{utils.RESET} ha abandonado la sala"}
193                 enviar_publico(aviso, sala)
194
195             usuarios_sala = {"tipo": "usuarios",
196                              "sala": sala,
197                              "lista": list(usuarios[sala].keys())}
198             enviar_publico(usuarios_sala, sala)
```

Imagen 18. Segunda parte del código que corresponde a la función “manejar_cliente()”.

Como segundo caso está el manejo de los mensajes públicos y privados, aquí dependiendo de la información de la petición es como se determina si un mensaje del cliente será público o privado, en caso de que el mensaje sea público se manda a llamar la función anteriormente explicada de “enviar_publico()” o en caso de que el mensaje sea privado se obtiene la información para identificar al cliente al que se le va a mandar el mensaje y aquí hay dos casos, uno para enviar el mensaje en caso de que el cliente al que le vamos a mandar el mensaje está dentro de la sala (aquí se manda a llamar la función “enviar_unicast()”), y el segundo caso para cuando el usuario al que le queremos mandar el mensaje no está conectado dentro de la sala, aquí se envía un mensaje de advertencia.

Como tercer caso tenemos cuando el cliente se quiere salir de la sala. Aquí, primero verificamos si el usuario esta actualmente en la sala y en caso de que si este, lo eliminamos

de la sala y se envía un aviso dentro del chat para informar que ese cliente salió de la sala. Adicionalmente a eso, se actualiza la lista de usuarios conectados en la sala, eliminando al cliente que se salió.

```
199
200         elif tipo == "AUDIOINFO" or tipo == "audio":
201             try:
202                 audio_sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
203                 audio_sock.bind(("0.0.0.0", 0))
204                 audio_port = audio_sock.getsockname()[1]
205
206                 respuesta = {"tipo": "READY",
207                              "port": audio_port}
208                 enviar_unicast(respuesta, addr)
209
210                 threading.Thread(target=recibir_audio_gbn, args=(audio_sock, addr, mensaje), daemon=True).start()
211             except Exception as e:
212                 print(f"Error iniciando transferencia de audio: {e}")
213                 error = {"tipo": "aviso",
214                          "sala": sala,
215                          "content": f">> {utils.RED}[system]{utils.RESET} No se pudo iniciar transferencia de audio."}
216                 enviar_unicast(error, addr)
```

Imagen 19. Tercera parte del código que corresponde a la función “manejar_cliente()”.

Como cuarto y ultimo caso tenemos cuando el cliente quiere iniciar una transferencia de audio dentro de la sala. Lo primero que hacemos es crear otro socket (UDP) adicional ligado a un puerto efímero para evitar congestionar el socket principal con los paquetes binarios del audio, nos aseguramos de que se haya creado bien e inmediatamente creamos un hilo que se va a encargar de la transferencia de los paquetes en segundo plano. En caso de que ocurra un error inesperado al momento de iniciar la transferencia, se enviará un mensaje indicando el error.

- **main()**

```
218
219 def main() -> None:
220     threading.Thread(target=manejar_cliente, daemon=True).start()
221
222     while True:
223         try:
224             time.sleep(1)
225         except KeyboardInterrupt:
226             print("\nServidor detenido.")
227             break
228
229 if __name__ == "__main__":
230     main()
```

Imagen 20. Código que corresponde a la función principal “main()”.

Parámetros:

- **No hay parámetros para esta función.**

Descripción de la función: En la función principal solo creamos el hilo principal que va a manejar el servidor y toda la lógica anteriormente mencionada. Mediante un ciclo "while" es que se va a mantener activo el servidor hasta que se indique lo contrario y se detenga el servidor.

5.- Archivo: cliente.py

Clase Cliente:

- **__init__(**self, usuario: str, sala: str**)**

```python
import socket
import threading
import json
import sys
from pathlib import Path

from audio import Audio
from stickers import obtener_sticker
import utils

SERVER = ("127.0.0.1", 5007)

class Cliente:
    def __init__(self, usuario: str, sala: str) -> None:
        self.usuario = usuario
        self.sala = sala
        self.activo = True
        self.audio = Audio()

        self.sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        self.sock.bind(('', 0))

        self.carpeta_script = Path(__file__).parent
        self.carpeta_sala = self.carpeta_script / f"{self.sala}"
        self.carpeta_sala.mkdir(parents=True, exist_ok=True)
        self.carpeta_usuario = self.carpeta_sala / f"{self.usuario}"
        self.carpeta_usuario.mkdir(parents=True, exist_ok=True)

        inicio = {"tipo": "inicio",
                  "user": self.usuario,
                  "sala": self.sala}
        self.sock.sendto(json.dumps(inicio).encode(), SERVER)
```

Imagen 21. Primera parte del código que corresponde a la clase "Cliente", definición del constructor de la clase "__init__()".

Parámetros:

- **self**
- **usuario: str**
  - Nombre de usuario del cliente.

- **sala: str**
    - Nombre de la sala en la que se encuentra el cliente.

Descripción del constructor: En esta parte del código se define el constructor, en el cual se definen todas las variables importantes que vamos a usar en esta clase "Cliente". Aquí también se crea el socket UDP para enviar y recibir mensajes, así como su puerto. Y, por último, se definen todas las rutas que serán esenciales para guardar los archivos de audio de cada cliente dentro de la sala.

- **recibir_mensaje(**self**)**

```
34      def recibir_mensaje(self) -> None:
35          while self.activo:
36              try:
37                  data, _ = self.sock.recvfrom(4096)
38                  mensaje = json.loads(data.decode())
39                  sala_mensaje = mensaje.get("sala", self.sala)
40                  tipo = mensaje.get("tipo")
41
42                  if sala_mensaje != self.sala:
43                      continue
44
45                  if tipo == "msj":
46                      if mensaje.get("privado"):
47                          print(f"{utils.YELLOW}[Privado de {mensaje['from']}]{utils.RESET}: {mensaje['content']}")
48                      else:
49                          print(f"{utils.BLUE}[{mensaje['user']}]{utils.RESET}: {mensaje['content']}")
50
51                  if tipo == "aviso":
52                      print(mensaje["content"])
53
54                  if tipo == "audio":
55                      if mensaje.get("privado"):
56                          print(f"{utils.YELLOW}[Privado de {mensaje['from']}]{utils.RESET}: {mensaje['content']}")
57                      else:
58                          print(f"{utils.BLUE}[{mensaje.get('from')}]{utils.RESET}: {mensaje['content']}")
59
60                  if tipo == "usuarios":
61                      print(f"Usuarios en sala {utils.MAGENTA}'{self.sala}'{utils.RESET}: {utils.MAGENTA}{', '.join(mensaje['lista'])}{utils.RESET}")
62              except:
63                  break
```

Imagen 22. Código que corresponde a la clase "Cliente", definición de la función "recibir_mensaje()".

Parámetro:

- **self**

Descripción de la función: Esta función se encarga de recibir los mensajes de los clientes dentro de la sala y para ello hay un bucle "while" que se ejecuta constantemente y espera la llegada de alguna petición de parte del servidor (un mensaje de cualquier cliente). Aquí hay diferentes casos dependiendo del mensaje o solicitud que llegue desde el servidor; el primero de ellos es cuando un cliente recibe un mensaje, el segundo caso es para avisos, el tercero es en caso de recibir un audio y el cuarto es un mensaje que indica la cantidad de clientes que hay conectados dentro de la sala.

Finalmente, es importante mencionar que, en caso de recibir mensajes o audios el programa identifica si son privados o públicos dependiendo de la información que le llegue desde el servidor.

- **enviar_mensaje(**self**)**

```python

    def enviar_mensaje(self) -> None:
        while self.activo:
            try:
                texto = input("").strip()

                if texto.lower() == "/salir":
                    salir = {"tipo": "salir",
                             "user": self.usuario,
                             "sala": self.sala}
                    self.sock.sendto(json.dumps(salir).encode(), SERVER)
                    self.activo = False
                    print(f">> {utils.RED}[system]{utils.RESET} Has salido de la sala")
                    self.sock.close()
                    break

                elif texto.lower() == "/audio":
                    audio_meta = {"tipo": "audio",
                                  "privado": False,
                                  "user": self.usuario,
                                  "sala": self.sala}
                    nombre_audio = self.audio.grabar_audio(audio_meta, self.sala, self.usuario)
                    print(f">> {utils.GREEN}[system]{utils.RESET} Has enviado el audio '{nombre_audio}'")
```

Imagen 23. Código que corresponde a la clase “Cliente”, primera parte de la función “enviar_mensaje()”.

Parámetro:

- **self**

Descripción: Esta función se encarga de enviar los mensajes de los clientes dentro de la sala y para ello hay un bucle “while” que se ejecuta constantemente y espera la llegada de alguna petición de parte del servidor (un mensaje de cualquier cliente). Aquí hay diferentes casos dependiendo del mensaje o solicitud que llegue desde el servidor y a continuación se explican cada uno de estos casos:

Para el primer caso, si uno de los clientes escribe en el chat el comando “**/salir**” entonces se va a guardar la información necesaria para enviarla al servidor como petición y este la interprete para realizar las acciones necesarias para sacarlo de la sala, mientras que, por el lado del cliente, a estos solo ser saldrá un mensaje indicando que el cliente que escribió ese comando salió de la sala.

Para el segundo caso, si el cliente escribe en el chat el comando “**/audio**” entonces se va a guardar la información necesaria y se va a ejecutar la función “grabar_audio()” para grabar el audio de 5 segundos, guardar el archivo en su respectiva ruta y obtener el nombre del archivo del audio para imprimirlo en la sala del chat para que los clientes lo puedan reproducir cuando quieran. Es importante mencionar que este caso solo aplica para los audios públicos, más adelante se explicara para el caso en el que sea privado.

```
87
88                  elif texto.lower().startswith("/sticker"):
89                      partes = texto.split(" ", 1)
90                      if len(partes) < 2:
91                          print(f">> {utils.RED}[system]{utils.RESET} Formato: /sticker nombre_sticker")
92                          continue
93                      _, nombre_sticker = partes
94                      sticker = obtener_sticker(nombre_sticker)
95                      if not sticker:
96                          print(f">> {utils.RED}[system]{utils.RESET} El sticker no existe")
97                          continue
98                      mensaje = {"tipo": "msj",
99                                 "privado": False,
100                                "user": self.usuario,
101                                "sala": self.sala,
102                                "content": sticker}
103                     self.sock.sendto(json.dumps(mensaje).encode(), SERVER)
104
105                 elif texto.lower().startswith("/reproducir"):
106                     partes = texto.split(" ", 1)
107                     if len(partes) < 2:
108                         print(f">> {utils.RED}[system]{utils.RESET} Formato: /reproducir nombre_archivo")
109                         continue
110                     _, nombre_archivo = partes
111                     ruta_audio = self.audio.buscar_audio(self.carpeta_sala, self.carpeta_usuario, self.usuario, nombre_archivo)
112                     if ruta_audio:
113                         self.audio.reproducir_audio(ruta_audio, nombre_archivo)
114                     else:
115                         print(f">> {utils.RED}[system]{utils.RESET} El audio no existe")
```

Imagen 24. Código que corresponde a la clase “Cliente”, segunda parte de la función “enviar_mensaje()”.

Para el tercer caso, si uno de los clientes escribe en el chat el comando “**/sticker [***nombre_del_sticker***]**” entonces se manda a llamar la función “obtener_sticker()” para obtener el sticker deseado dependiendo del nombre que el cliente haya indicado en el comando, pero en caso de que el nombre del sticker sea incorrecto o no exista un sticker bajo ese nombre, entonces en el chat se mostrará un mensaje de error. Una vez obtenido el sticker, se guarda toda la información necesaria para que el servidor pueda interpretar la petición y se envía al servidor.

Para el cuarto caso, si uno de los clientes escribe en el chat el comando “**/reproducir [***nombre_del_audio.wav***]**” entonces se manda a llamar la función “buscar_audio()” para obtener la ruta del audio que el cliente quiere reproducir y posteriormente, si se encontró el audio en la ruta, entonces ahora se manda a llamar la función “reproducir_audio()” para generar la interfaz gráfica y así cualquier cliente pueda escuchar el audio. En caso de que el audio no exista o el cliente escriba mal el nombre del archivo, entonces manda un mensaje de error en el chat.

```
116
117                 elif texto.startswith("@"):
118                     partes = texto.split(" ", 1)
119                     if len(partes) < 2:
120                         print(f">> {utils.RED}[system]{utils.RESET} Formato: @usuario mensaje")
121                         continue
122
123                     if partes[1].startswith("/sticker"):
124                         partes2 = partes[1].split(" ")
125                         if len(partes2) < 2:
126                             print(f">> {utils.RED}[system]{utils.RESET} Formato: /sticker nombre_sticker")
127                             continue
128                         _, nombre_sticker = partes2
129                         sticker = obtener_sticker(nombre_sticker)
130                         if not sticker:
131                             print(f">> {utils.RED}[system]{utils.RESET} El sticker no existe")
132                             continue
133                         destinatario, _ = partes
134                         destinatario = destinatario[1:]
135                         mensaje = {"tipo": "msj",
136                                    "privado": True,
137                                    "from": self.usuario,
138                                    "to": destinatario,
139                                    "content": sticker,
140                                    "sala": self.sala}
141                         self.sock.sendto(json.dumps(mensaje).encode(), SERVER)
142                         print(f"{utils.ORANGE}[Tú -> {destinatario}]{utils.RESET}: {sticker}")
143                         continue
```

Imagen 25. Código que corresponde a la clase “Cliente”, tercera parte de la función “enviar_mensaje()”.

```
144
145                 elif partes[1] == "/audio":
146                     destinatario, _ = partes
147                     destinatario = destinatario[1:]
148                     audio_meta = {"tipo": "audio",
149                                   "privado": True,
150                                   "from": self.usuario,
151                                   "to": destinatario,
152                                   "sala": self.sala}
153                     nombre_audio = self.audio.grabar_audio(audio_meta, self.sala, self.usuario)
154                     print(f"{utils.ORANGE}[Tú -> {destinatario}]{utils.RESET}: has enviado el audio '{nombre_audio}'")
155                     continue
156
157                 else:
158                     destinatario, contenido = partes
159                     destinatario = destinatario[1:]
160                     mensaje = {"tipo": "msj",
161                                "privado": True,
162                                "from": self.usuario,
163                                "to": destinatario,
164                                "content": contenido,
165                                "sala": self.sala}
166                     self.sock.sendto(json.dumps(mensaje).encode(), SERVER)
167                     print(f"{utils.ORANGE}[Tú -> {destinatario}]{utils.RESET}: {contenido}")
168                     continue
```

Imagen 26. Código que corresponde a la clase “Cliente”, cuarta parte de la función “enviar_mensaje()”.

Para el quinto caso, si uno de los clientes escribe el comando “**@[***nombre_del_cliente***] [***mensaje***]**” entonces el programa identifica que se trata de un mensaje privado y este mensaje solo se va a enviar al cliente al que se haga referencia después del “@”. Este caso se divide en otros tres casos diferentes, uno para stickers privados, otro para audios privados y el ultimo

para mensajes privados, sin embargo, estos casos funcionan bajo la misma lógica que con los stickers, audios y mensajes públicos, la única diferencia es que aquí al momento de construir la metainformación de cada petición se indica que será privado de tal modo que el dato del diccionario "privado" será "True" en vez de "False" (para mensajes públicos). Al indicar ese dato en la metainformación el servidor reconoce esta petición como un mensaje privado y realiza unicast para el cliente al que se le quiere enviar el mensaje privado.

```
169
170                 else:
171                     mensaje = {"tipo": "msj",
172                                "privado": False,
173                                "user": self.usuario,
174                                "sala": self.sala,
175                                "content": texto}
176                     self.sock.sendto(json.dumps(mensaje).encode(), SERVER)
177             except KeyboardInterrupt:
178                 salir = {"tipo": "salir",
179                          "user": self.usuario,
180                          "sala": self.sala}
181                 self.sock.sendto(json.dumps(salir).encode(), SERVER)
182                 self.activo = False
183                 self.sock.close()
184                 sys.exit(0)
```

Imagen 27. Código que corresponde a la clase "Cliente", quinta parte de la función "enviar_mensaje()".

Como sexto y ultimo caso, si uno de los clientes no escribe ningún comando y en cambio, solo escribe un mensaje sin formato, el programa identificará que se trata de un mensaje publico y solo se va a construir la metainformación para mandarla al servidor.

Por último, tenemos la excepción del bloque "try" el cual contiene todo el código que se acaba de explicar. Como excepción esta que si el cliente decide interrumpir la ejecución del programa entonces se va a mandar la petición al servidor para salir de la sala inmediatamente y en consecuencia se va a cerrar el socket, terminando el proceso.

- **iniciar(**self**)**

```
185
186     def iniciar(self) -> None:
187         threading.Thread(target=self.recibir_mensaje, daemon=True).start()
188         self.enviar_mensaje()
189
```

Imagen 28. Código que corresponde a la clase "Cliente", definición de la función "iniciar()".

Parámetro:

- **self**

Descripción de la función: En esta función se inicializan los hilos, Un hilo para recibir los mensajes en segundo plano y un hilo principal para enviar los mensajes respectivamente. Una vez inicializados los hilos, se manda a llamar la función “enviar_mensaje()”.

- **obtener_salas()**

```
191 def obtener_salas() -> list:
192     temp_sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
193     temp_sock.bind(('', 0))
194     peticion = {"tipo": "listar_salas"}
195     temp_sock.sendto(json.dumps(peticion).encode(), SERVER)
196     data, _ = temp_sock.recvfrom(4096)
197     temp_sock.close()
198     try:
199         respuesta = json.loads(data.decode())
200         return respuesta["lista"] if respuesta["tipo"] == "salas" else []
201     except:
202         return []
203
```

Imagen 29. Código que corresponde a la función “obtener_salas()”.

Parámetros:

- **No hay parámetros para esta función.**

Descripción de la función: En esta función lo que se hace primero es crear un socket UDP temporal para solicitar las salas disponibles mediante una petición al servidor y una vez recibida lista de las salas, se muestran las salas disponibles al cliente.

- **main()**

```python

def main() -> None:
    print(f">> {utils.RED}[system]{utils.RESET} Obteniendo salas disponibles...\n")
    salas_disponibles = obtener_salas()
    if salas_disponibles:
        print("Salas disponibles:")
        for i, s in enumerate(salas_disponibles, 1):
            print(f"{i}. {s}")
    else:
        print(f">> {utils.RED}[system]{utils.RESET} No hay salas actvas. Se creará una nueva sala al unirte")

    sala = input("\nEscribe el nombre de la sala a la que desesar unirte (o crea una nueva): ").strip()
    if not sala:
        sala = "general"

    usuario = input("Ingresa tu nombre de usuario: ")

    utils.limpiar_terminal()
    print(f"""
 ===================================
  BIENVENIDO A LA SALA '{sala}'
 ===================================
  USUARIO: {usuario}
 =======================
 """)
    cliente = Cliente(usuario, sala)
    cliente.iniciar()

if __name__ == "__main__":
    main()
```

Imagen 30. Código que corresponde a la función principal “main()”.

Parámetros:

- **No hay parámetros para esta función.**

Descripción de la función principal: En esta función lo que se hace primero obtener el listado de salas disponibles para el cliente mediante la función “obtener_salas()”. Si hay salas disponibles entonces se muestran las salas al cliente, pero en caso de que algún cliente no haya creado alguna sala anteriormente entonces deberá crear una sala nueva, para ello deberá escribir el nombre de la sala para crearla, de lo contrario, si no escribe algún nombre y da “enter” entonces entrará automáticamente a la sala general. Posteriormente, para entrar a una sala, el cliente tiene que introducir su nombre de usuario para ser aceptado en la sala que haya creado o la sala general.

Una vez que el cliente haya entrado a la sala, se crea una instancia de la clase “Cliente” y se manda a llamar la función “iniciar()” que va a inicializar los hilos para enviar y recibir mensajes mencionados anteriormente.

CONCLUSIÓN

- **García Escamilla Bryan Alexis**

Durante el desarrollo de la práctica se presentó un problema que cambio el flujo de todo el programa. En un inicio y como indica la práctica, se tendrían que haber usado sockets de datagrama multicast, esto para facilitar el envío de mensajes a todos los usuarios dentro de

una sala (grupo multicast) y así evitar enviar el mismo mensaje a cada uno de los usuarios en la sala al usar sockets de datagrama unicast, sin embargo el problema surgió al momento de implementar el envío de mensajes privados, ya que no fue posible filtrar el mensaje enviado al grupo para que solo le llegara al usuario correspondiente, esto causaba varios problemas: los mensajes "privados" se enviaban a todos los usuarios en la sala, el servidor y el cliente entraban en un bucle infinito, en el que se enviaba el mismo mensaje infinitamente, entre otros. Por lo que al final se optó por implementar sockets de datagrama unicast, ya que estos nos permiten enviar mensajes directos a cualquier usuario en la sala, mientras que para enviar mensajes a toda la sala se tuvo que enviar el mismo mensaje a cada uno de los usuarios en la sala, aunque esto consume más ancho de banda, ofrece una conexión más directa y confiable.

Por otro lado, para la grabación, envío y reproducción de los audios fue necesario la creación de una carpeta por cada sala durante la ejecución del programa, así como la creación de una carpeta por cada usuario que ingresaba a cualquier sala. Esto permitió manejar de forma correcta la reproducción de los audios. Todos los usuarios pueden escuchar los audios enviados a la sala, pero no todos pueden escuchar los audios privados si no que solo el quien los envía y recibe puede hacerlo, así que, para cada audio grabado, en su nombre se incluye el nombre del usuario quien lo envía y este se guarda en la carpeta correspondiente, ya sea en la carpeta de la sala (audio público) o dentro de la carpeta del destinatario (audio privado).

De esta manera si alguien quiere reproducir un audio enviado a la sala, solo se busca dentro de la carpeta de la sala actual aquel con el nombre del audio a reproducir, mientras que para reproducir un audio enviado a un usuario especifico, se busca en cada una de las carpetas de los usuarios dentro de la sala aquel que tenga en el nombre el nombre del usuario emisor y por último para reproducir un audio recibido de forma privada únicamente se busca en la carpeta del usuario actual.

Por último, para el envío de los audios se empleó el algoritmo Go-Back-N para enviar el audio desde el cliente al servidor por fragmentos de bytes (paquetes), para ello se tomó el audio guardado en memoria después de grabarlo y evitar hacer uso de una copia de este. También hubo la necesidad de crear una clase por separado para manejar todo lo relacionado a los audios para que de esta manera la clase del cliente no fuera innecesariamente larga. Y para evitar el timeout en el servidor al haber recibido todos los paquetes desde el cliente, este último le envía un mensaje cuando inicia el envío y cuando termina, para que el servidor desactive el timeout.

- **Meléndez Macedonio Rodrigo**

Para el desarrollo de esta practica tuvimos muchos problemas al inicio, pues inicialmente pensábamos crear una interfaz gráfica de usuario para representar el chat, sin embargo, a medida que íbamos desarrollando el código, vimos que se iba haciendo cada vez más extenso y complejo, por lo que da idea de crear una interfaz gráfica quedo descartada rápidamente, pues no iba a dar tiempo para crear una interfaz para todo lo que teníamos que implementar en el código. Es por esa razón que mejor decidimos desarrollar esta práctica solo para su

implementación en la terminal, de esta manera se nos facilito más pues ya no teníamos que preocuparnos por otros aspectos que se podrían considerar hasta innecesarios.

Sin embargo, en el momento surgieron varias dudas que pronto solucionamos, pues en su momento no sabíamos muy bien como implementar los stickers a la consola. Fue gracias a la idea de un amigo que nos facilito la implementación de esta característica a la consola ya que lo único que hicimos fue crear "stickers" mediante una cadena de caracteres y de esta manera, cada vez que un cliente solicite algún sticker, este solo tendrá que escribir el comando respectivamente para solicitarlo y enviarlo. Otra de nuestras dudas en un inicio era respecto a los audios, pues de la misma forma que con los stickers, en la consola no podemos reproducir audios directamente y fui ahí cuando decidimos implementar un sistema de carpetas que se asignaría a cada cliente que entre a la sala, de esta forma sería fácil identificar cada audio creado por cada cliente, pues gracias a la ayuda de bibliotecas como "wavio", "sounddevice", "tkinter" y "pygame" fue que logramos guardar los archivos de audio bajo la extensión ".wav" en cada carpeta y a partir de ahí, cada vez que un cliente quiera reproducir un audio, deberá identificar el nombre el audio mandado por otro cliente y cuando lo haga correctamente, se abrirá una pequeña interfaz gráfica que le permitirá al cliente reproducir el audio.

A la par que íbamos solucionando esos problemas, de repente surgió otro que era aún más problemático, pues nos dimos cuenta de que había muchas limitaciones al momento de usar multicast en Python y eso nos detuvo por un momento, a tal punto de que nos planteamos hacer la practica en otro lenguaje de programación, sin embargo, llegamos a la conclusión de que lo mejor sería usar sockets de datagrama unicast en vez de multicast. Al no usar multicast, nuestra solución fue utilizar unicast dentro de un bucle para enviar los mensajes a todos los clientes de la sala en caso de que fueran mensajes públicos, y para el caso de los mensajes privados, solo se enviaría mediante unicast a un único cliente (sin bucle).

La verdad, durante el desarrollo de la practica se presentaron muchos problemas y eso hizo que la practica se hiciera bastante pesada de realizar, pero al final logramos terminarla bien y quedamos satisfechos con el resultado a pesar de las dificultades.